COSC 611 Big Data Analytics — Midterm Take-Home Assignment

Department of Computer Science, Morgan State University

Student Name: _____ Student ID: _____

October 2026

- Overview
- Question 1 — Similarity Measures
  - 1.1 What is a similarity measure?
  - 1.2 Why are similarity measures necessary?
  - 1.3 Example data
  - 1.4 Types of similarity measures, with computations
    - (a) Euclidean distance (L2)
    - (b) Manhattan distance (L1, city-block)
    - (c) Cosine similarity
    - (d) Pearson correlation coefficient
    - (e) Jaccard similarity (sets)
  - 1.5 Computation on Spark
  - 1.6 Interpretation: the choice of measure matters
- Question 2 — Hypothesis Testing
  - 2.1 What is hypothesis testing?
  - 2.2 Distinguishing the χ^2, t and F tests
  - 2.3 Chi-square test of independence
  - 2.4 Student's t-test
  - 2.5 F-test
    - (a) F-test for equality of two variances
    - (b) One-way ANOVA F-test (comparing 3 or more means)
  - 2.6 Summary of results
- Question 3 — Supervised vs Unsupervised Learning, and Clustering
  - 3.1 Supervised versus unsupervised learning
  - 3.2 Example data
  - 3.3 Algorithm 1 — K-Means (partitional, hard clustering)
  - 3.4 Algorithm 2 — Bisecting K-Means (hierarchical, divisive)
  - 3.5 Algorithm 3 — Gaussian Mixture Model (probabilistic, soft clustering)
  - 3.6 Comparison of the three algorithms
- Question 4 — Classification
  - 4.1 What is classification?
  - 4.2 The three algorithms in words

    - 4.3 Worked example A — “Play Tennis” (hand computation + Spark)
        - Naïve Bayes by hand
        - Random Forest building block: Gini split selection, by hand and on Spark
        - SVM by hand on a tiny 2-D set
    - 4.4 Worked example B — spam detection with all three classifiers on Spark
    - 4.5 Distinguishing Naïve Bayes, SVM and Random Forest
- References

Overview

This report answers the four midterm questions. Every concept is illustrated with (a) a small example dataset, (b) a hand computation that can be checked with a calculator, and (c) the same computation carried out on **Apache Spark** (PySpark, `pyspark.ml` / Spark MLlib). The Spark programs are submitted alongside this report:

Question	Spark program	Captured output
Q1 Similarity measures	`code/q1_similarity.py`	`outputs/q1_similarity.txt`
Q2 Hypothesis testing	`code/q2_hypothesis_tests.py`	`outputs/q2_hypothesis_tests.txt`
Q3 Clustering	`code/q3_clustering.py`	`outputs/q3_clustering.txt`
Q4 Classification	`code/q4_classification.py`	`outputs/q4_classification.txt`

Environment used to produce the outputs: Apache Spark 4.0.1 (PySpark) in `local[*]` mode, Python 3.13, OpenJDK 21, SciPy 1.18 (used only to cross-check p-values). Each script runs with `spark-submit code/qN_*.py` or `python3 code/qN_*.py`. All numbers quoted below are copied from the captured outputs.

Question 1 — Similarity Measures

1.1 What is a similarity measure?

A **similarity measure** is a function $s(x, y)$ that takes two objects (two customers, two documents, two images, two gene sequences, …) and returns a number that says *how alike they are*. The higher the value, the more alike the objects are. Its opposite is a **distance (dissimilarity) measure** $d(x, y)$, where a *smaller* value means more alike. One can usually be turned into the other, for example $s = \frac{1}{1+d}$ or $s = 1 - d$ when $d \in [0, 1]$.

A good similarity measure usually has these properties:

- **Bounded/normalised**, often in $[0, 1]$ or $[-1, 1]$.
- **Symmetric**: $s(x, y) = s(y, x)$.
- **Maximal for identical objects**: $s(x, x)$ is the largest possible value.

A *distance metric* must also satisfy non-negativity, $d(x, y) = 0 \Leftrightarrow x = y$, and the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$.

1.2 Why are similarity measures necessary?

Many Big Data analytics tasks come down to the question *"which items are close to which?"*:

- **Recommender systems.** Collaborative filtering finds users whose rating vectors are similar to yours and recommends what they liked ("people like you also bought …").
- **Clustering.** K-Means, hierarchical clustering and DBSCAN group points that are close to each other (Question 3), so they need a distance.
- **Classification.** k-Nearest-Neighbours labels a point by the labels of its most similar neighbours. SVM kernels are similarity functions.
- **Information retrieval and search.** Search engines rank documents by their cosine similarity to the query vector (TF-IDF).
- **De-duplication and plagiarism detection.** Near-duplicate web pages and records are found with Jaccard similarity and MinHash.
- **Anomaly detection.** A point that is dissimilar to everything else is an outlier.

The right measure depends on the data type: numeric vectors, sets, text, or binary attributes. A wrong choice gives misleading neighbours, as the example below shows. At Big Data scale, comparing all n^2 pairs is too expensive, so Spark provides **Locality-Sensitive Hashing (LSH)**. LSH finds similar pairs approximately without comparing every pair.

1.3 Example data

Five users rated five movies on a 0–5 scale (0 = not watched):

User	Inception	Titanic	Avatar	Matrix	Frozen
Alice	5	3	0	4	1
Bob	4	0	0	5	1
Carol	1	5	4	0	5
Dave	5	4	1	4	0
Eve	0	5	5	1	4

In the hand computations below, A = Alice = (5, 3, 0, 4, 1) and D = Dave = (5, 4, 1, 4, 0).

1.4 Types of similarity measures, with computations

(a) Euclidean distance (L2)

The straight-line distance between two points:

$$d_E(x,y) = \sqrt{\sum_i (x_i - y_i)^2}, \qquad s_E = \frac{1}{1 + d_E}$$

$A - D = (0, -1, -1, 0, 1)$, so $d_E = \sqrt{0 + 1 + 1 + 0 + 1} = \sqrt{3} = 1.7321$ and $s_E = 1/(1 + 1.7321) = 0.3660$.

Euclidean distance is sensitive to scale and magnitude. It is a natural fit for dense, low-dimensional numeric data such as K-Means features.

(b) Manhattan distance (L1, city-block)

$$d_M(x,y) = \sum_i |x_i - y_i|$$

$d_M(A,D) = 0 + 1 + 1 + 0 + 1 = 3$. Manhattan distance is less affected by one large difference than Euclidean distance, so it is more robust to outliers.

(c) Cosine similarity

The cosine of the angle between two vectors. It measures orientation rather than magnitude:

$$\cos(x,y) = \frac{x \cdot y}{\|x\| \ \|y\|}$$

- $A \cdot D = 25 + 12 + 0 + 16 + 0 = 53$
- $\|A\| = \sqrt{25 + 9 + 0 + 16 + 1} = \sqrt{51} = 7.1414$
- $\|D\| = \sqrt{25 + 16 + 1 + 16 + 0} = \sqrt{58} = 7.6158$
- $\cos(A,D) = 53/(7.1414 \times 7.6158) = 53/54.387 = 0.9745$

The range is $[-1, 1]$, or $[0, 1]$ for non-negative data. Cosine is the standard measure for text (TF-IDF vectors) and sparse high-dimensional data. A long document and a short one about the same topic still get a high score.

(d) Pearson correlation coefficient

Cosine similarity applied after **mean-centring** each vector. This removes each user's personal rating bias (a "harsh" rater versus a "generous" one):

$$r(x, y) = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}}$$

- $\bar{A} = 13/5 = 2.6$ and $\bar{D} = 14/5 = 2.8$
- Deviations: $A' = (2.4, 0.4, -2.6, 1.4, -1.6)$ and $D' = (2.2, 1.2, -1.8, 1.2, -2.8)$
- $\sum A'D' = 5.28 + 0.48 + 4.68 + 1.68 + 4.48 = 16.6$
- $\sum A'^2 = 17.2$ and $\sum D'^2 = 18.8$
- $r = 16.6/\sqrt{17.2 \times 18.8} = 16.6/17.982 = 0.9231$

The range is $[-1, 1]$, where -1 means opposite tastes. Alice versus Eve gives $r = -0.82$: their tastes are strongly opposed.

(e) Jaccard similarity (sets)

Jaccard similarity is used when the data is a **set** or binary presence/absence (which movies were watched, which words appear):

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

- Alice watched {Inception, Titanic, Matrix, Frozen}.
- Dave watched {Inception, Titanic, Avatar, Matrix}.
- Intersection = 3 and union = 5, so $J = 3/5 = 0.6$.
- Bob watched {Inception, Matrix, Frozen}, which is a subset of Alice's set, so $J(A, B) = 3/4 = 0.75$.

Other measures for binary data include the **Simple Matching Coefficient**, which also counts shared 0s, the **Hamming distance**, which counts positions that differ, and the **Dice coefficient** $2|S \cap T|/(|S| + |T|)$. For text there is also **edit (Levenshtein) distance**.

1.5 Computation on Spark

`code/q1_similarity.py` loads the rating matrix into a Spark DataFrame of `Vectors.dense` features. It then **self cross-joins** the DataFrame to form every user pair and evaluates each measure on the pair. This is the distributed form of the n^2 comparison.

```
        a, b = df.alias("a"), df.alias("b")
        pairs = a.crossJoin(b).where(F.col("a.user") < F.col("b.user"))
        result = (pairs.withColumn("euclidean", euclidean("x", "y"))
                       .withColumn("cosine", cosine("x", "y"))
                       .withColumn("pearson", pearson("x", "y"))
                       .withColumn("jaccard", jaccard("x", "y")) ...)
```

Spark output (all 10 pairs):

u1	u2	Euclidean	1/(1+d)	Manhattan	Cosine	Pearson	Jaccard
Alice	Bob	3.3166	0.2317	5	0.8859	0.7711	**0.75**
Alice	Carol	8.2462	0.1082	18	0.4277	−0.7197	0.60
Alice	Dave	**1.7321**	**0.3660**	**3**	**0.9745**	**0.9231**	0.60
Alice	Eve	8.4853	0.1054	18	0.3935	−0.8225	0.60
Bob	Carol	9.5394	0.0949	21	0.1697	−0.9545	0.40
Bob	Dave	4.4721	0.1827	8	0.8104	0.5901	0.40
Bob	Eve	9.5394	0.0949	21	0.1697	−0.9545	0.40
Carol	Dave	8.1854	0.1089	17	0.4652	−0.6392	0.60
Carol	Eve	2.0000	0.3333	4	0.9701	0.9091	0.60
Dave	Eve	8.1854	0.1089	17	0.4652	−0.6392	0.60

The Spark values match the hand computations for Alice–Dave exactly.

The script also shows three **Spark-native** ways of computing these measures:

1. **Cosine via `Normalizer`.** `Normalizer(p=2.0)` scales every vector to unit length, so a plain dot product of two unit vectors *is* the cosine. Spark gives the same values, for example Alice–Dave = 0.9745 and Carol–Eve = 0.9701.

2. **Pearson via `pyspark.ml.stat.Correlation.corr`.** This computes the full correlation matrix in one distributed pass. (Spark correlates columns, so the matrix is transposed so that each user is a column.)

```
          Alice     Bob   Carol    Dave     Eve
Alice     1.000   0.771  -0.720   0.923  -0.823
  Bob     0.771   1.000  -0.955   0.590  -0.955
Carol    -0.720  -0.955   1.000  -0.639   0.909
 Dave     0.923   0.590  -0.639   1.000  -0.639
  Eve    -0.823  -0.955   0.909  -0.639   1.000
```

3. **Jaccard at scale via `MinHashLSH`.** MinHash hashes each set so that the probability that two sets get the same hash *equals* their Jaccard similarity. `approxSimilarityJoin` then compares only pairs

that land in the same bucket, instead of all n^2 pairs. With 50 hash tables, the approximate Jaccard matched the exact value for every pair here (for example Alice–Bob 0.75 and Bob–Carol 0.40). On millions of users, this is what makes the computation feasible.

Text example. The script also tokenises three short documents (`Tokenizer` + `CountVectorizer`):

pair	Cosine	Jaccard
“big data analytics with spark” vs “spark makes big data processing fast”	0.5477	0.375
either of those vs “the cat sat on the mat”	0.0	0.0

The first pair shares 3 words (*big*, *data*, *spark*), so $J = 3/8 = 0.375$ and $\cos = 3/\sqrt{5 \cdot 6} = 0.5477$.

1.6 Interpretation: the choice of measure matters

The question “who is most like Alice?” gets different answers. Euclidean, Manhattan, cosine and Pearson all choose **Dave**, because Dave’s ratings have the same values and the same pattern. Jaccard chooses **Bob** (0.75), because Jaccard ignores the rating values and only looks at *which* movies were watched. In general:

- Use **cosine** for text and sparse vectors.
- Use **Pearson** for ratings with user bias.
- Use **Euclidean or Manhattan** for dense numeric features, after scaling them.
- Use **Jaccard** for sets or binary data.

Question 2 — Hypothesis Testing

2.1 What is hypothesis testing?

Hypothesis testing is a formal procedure for using **sample data** to decide whether a claim about a **population** is supported. Analysts usually only see a sample, so the question is whether an observed effect is real or could just be random sampling noise. The steps are:

1. **State the hypotheses.** The *null hypothesis* H_0 is the "no effect / no difference / independent" claim. The *alternative hypothesis* H_1 is what we suspect is true.
2. **Choose the significance level** α, usually 0.05. This is the probability of rejecting H_0 when it is actually true, called a **Type I error** (false positive). Failing to reject a false H_0 is a **Type II error** (false negative). The power of a test is $1 - P$(Type II).
3. **Choose the test and compute the test statistic** (χ^2, t or F) from the sample.
4. **Find the p-value**: the probability, *assuming* H_0 *is true*, of seeing a statistic at least as extreme as the one observed. Equivalently, compare the statistic with a **critical value** from the statistic's distribution.
5. **Decide.** If $p < \alpha$, reject H_0 (the result is statistically significant). Otherwise, *fail to reject* H_0. Failing to reject does not prove H_0 true.

Tests can be **one-tailed** or **two-tailed**, depending on whether H_1 specifies a direction.

2.2 Distinguishing the χ^2, t and F tests

	Chi-square (χ^2) test	**Student's t-test**	**F-test**
Question answered	Are two *categorical* variables independent? Does an observed frequency distribution fit an expected one?	Is a *mean* different from a value, or are two *means* different?	Are two *variances* equal? Are 3 or more means equal (ANOVA)?
Data type	Counts/frequencies of categories	Continuous, approximately normal	Continuous, approximately normal
Statistic	$\chi^2 = \sum \frac{(O-E)^2}{E}$	$t = \frac{\bar{x}_1 - \bar{x}_2}{s_p\sqrt{1/n_1 + 1/n_2}}$	$F = \frac{s_1^2}{s_2^2}$, or $F = \frac{MS_{between}}{MS_{within}}$
Distribution	χ^2 with $(r-1)(c-1)$ df	t with $n_1 + n_2 - 2$ df (two-sample) or $n - 1$ df (one-sample)	F with (d_1, d_2) df

	Chi-square (χ^2) test	Student's t-test	F-test
Shape	Right-skewed, values ≥ 0	Symmetric bell, heavier tails than the normal	Right-skewed, values ≥ 0
Parametric?	Non-parametric (no normality assumption)	Parametric	Parametric
Typical use	Survey answers vs demographic; click vs device	A/B test of average spend; before/after	Comparing variability of processes; ANOVA across many groups; overall significance of a regression
Spark support	`pyspark.ml.stat.ChiSquareTest` (built in)	No built-in test. Computed from distributed aggregates (mean, var, count)	`ANOVATest` and `FValueTest` (`org.apache.spark.ml.stat`), or from aggregates

How they relate. t^2 with v df equals F with $(1, v)$ df, so a two-group ANOVA is the same test as a two-sample t-test. The χ^2 distribution is the distribution of a sum of squared standard normals, and F is a ratio of two scaled χ^2 variables.

2.3 Chi-square test of independence

Example. An online store records 200 customers' device and whether they made a purchase.

Observed	Purchase = Yes	Purchase = No	Row total
Mobile	30	70	100
Desktop	45	55	100
Column total	75	125	N = 200

H_0 : device and purchase are independent. H_1 : they are associated.

Hand computation. The expected count for each cell is $E = \frac{\text{row total} \times \text{column total}}{N}$:

- $E_{Mobile,Yes} = 100 \cdot 75/200 = 37.5$ and $E_{Mobile,No} = 62.5$
- The Desktop row has the same expected counts.

$$\chi^2 = \frac{(30-37.5)^2}{37.5} + \frac{(70-62.5)^2}{62.5} + \frac{(45-37.5)^2}{37.5} + \frac{(55-62.5)^2}{62.5} = 1.5 + 0.9 + 1.5 + 0.9 = 4.8$$

The degrees of freedom are $(2-1)(2-1) = 1$, and the critical value is $\chi^2_{0.05,\,1} = 3.8415$. Since $4.8 > 3.8415$, **reject** H_0 : purchase rate depends on the device (30% on mobile versus 45% on desktop).

Spark. `DataFrame.crosstab` builds the table, and a groupBy/join pipeline reproduces the $(O-E)^2/E$ contributions (0.9, 1.5, 0.9, 1.5). The built-in MLlib test is then run on a numeric label and a feature

vector:

```
        r = ChiSquareTest.test(enc, "features", "label").head()
        # Spark ChiSquareTest: statistic = 4.8000, df = 1, p-value = 0.0285
        # scipy check:          statistic = 4.8000, p-value = 0.0285
```

$p = 0.0285 < 0.05$, so we reject H_0.

2.4 Student's t-test

Example. Ten students were taught with Method A and ten with Method B. Their exam scores were:

- A: 78, 82, 85, 88, 75, 80, 84, 86, 79, 83
- B: 72, 75, 78, 70, 74, 77, 73, 76, 71, 74

$H_0 : \mu_A = \mu_B$. $H_1 : \mu_A \neq \mu_B$ (two-tailed, equal variances assumed).

Hand computation.

- $\bar{x}_A = 82$, $s_A^2 = 16$; $\bar{x}_B = 74$, $s_B^2 = 6.667$
- Pooled variance: $s_p^2 = \frac{9(16) + 9(6.667)}{18} = 11.333$
- Standard error: $SE = \sqrt{11.333\,(1/10 + 1/10)} = 1.5055$
- $t = (82 - 74)/1.5055 = 5.3137$, with $df = 18$

The critical value is $t_{0.025,\,18} = 2.1009$. Since $|t| > 2.1009$, **reject** H_0: Method A gives significantly higher scores.

Spark. MLlib has no t-test class, but the test only needs the count, mean and sample variance of each group. These are computed in **one distributed aggregation**, which scales to billions of rows. Only the final scalar formula and the t-distribution lookup run on the driver.

```
        summ = scores.groupBy("method").agg(F.count("score"), F.mean("score"),
                                            F.var_samp("score"),
F.stddev_samp("score"))
        # Spark-computed t = 5.3137, df = 18, two-tailed p-value = 0.000047
        # scipy check:      t = 5.3137, p-value = 0.000047
```

The script also runs a **one-sample t-test** ($H_0 : \mu_A = 75$): $t = 5.534$, $df = 9$, $p = 0.00036$, so we reject H_0.

2.5 F-test

(a) F-test for equality of two variances

Example. Two bottling machines each fill 10 bottles (in ml):

- Machine X: 502, 498, 510, 495, 505, 490, 512, 497, 508, 493
- Machine Y: 500, 501, 499, 502, 498, 500, 501, 499, 503, 497

$H_0 : \sigma_X^2 = \sigma_Y^2$. $H_1 : \sigma_X^2 \neq \sigma_Y^2$.

The sample variances are $s_X^2 = 57.111$ and $s_Y^2 = 3.333$, so $F = 57.111/3.333 = 17.13$ with $df = (9, 9)$. The upper critical value is $F_{0.025;\,9,\,9} = 4.026$. Since $F > 4.026$, **reject** H_0 : Machine X is far less consistent. Both machines have almost the same *mean* (501 vs 500 ml), so a t-test would miss this problem. The F-test catches it.

Spark computes `var_samp` per group in a single aggregation and gives F = 17.1333 with p = 0.000245.

(b) One-way ANOVA F-test (comparing 3 or more means)

Example. Crop yield for three fertilisers, with 5 plots each:

- F1: 20, 22, 19, 24, 25 (mean 22)
- F2: 28, 30, 27, 26, 29 (mean 28)
- F3: 18, 20, 22, 19, 21 (mean 20)

$H_0 : \mu_1 = \mu_2 = \mu_3$. H_1 : at least one mean differs.

Hand computation. The grand mean is 23.333.

- $SSB = 5[(22 - 23.33)^2 + (28 - 23.33)^2 + (20 - 23.33)^2] = 173.33$
- $SSW = 26 + 10 + 10 = 46$
- $MSB = 173.33/2 = 86.667$ and $MSW = 46/12 = 3.833$
- $F = 86.667/3.833 = 22.61$, with $df = (2, 12)$

The critical value is $F_{0.05;\,2,\,12} = 3.885$, so **reject** H_0 .

Spark. The ANOVA table is computed with DataFrame aggregations, and Spark's `org.apache.spark.ml.stat.ANOVATest` is also called. That class has no Python wrapper in PySpark 4.0, so the script calls it through the JVM gateway:

```
        jdf = spark._jvm.org.apache.spark.ml.stat.ANOVATest.test(anova_df._jdf,
"features", "label")
        # Spark ANOVATest: F = 22.6087, df = (2, 12), p-value = 0.000085
        # scipy check:     F = 22.6087, p-value = 0.000085
```

2.6 Summary of results

Test	Statistic	df	p-value	Critical value (α=0.05)	Decision
χ^2 independence	4.80	1	0.0285	3.84	Reject H_0

Test	Statistic	df	p-value	Critical value (α=0.05)	Decision
Two-sample t	5.31	18	0.000047	±2.10	Reject H_0
F (variances)	17.13	(9, 9)	0.000245	4.03	Reject H_0
ANOVA F	22.61	(2, 12)	0.000085	3.89	Reject H_0

Question 3 — Supervised vs Unsupervised Learning, and Clustering

3.1 Supervised versus unsupervised learning

	Supervised learning	Unsupervised learning
Training data	**Labelled**: each example x comes with the correct answer y	**Unlabelled**: only x is given
Goal	Learn a mapping $f: x \rightarrow y$ that predicts labels or values for new data	Discover hidden *structure*: groups, density, low-dimensional representation, associations
Feedback	An error signal against known labels (loss)	No "right answer". Quality is judged by internal criteria (cohesion and separation) or by a domain expert
Typical tasks	Classification (spam/not spam), regression (house price)	Clustering (customer segments), dimensionality reduction (PCA), association rules (market basket), anomaly detection
Example algorithms	Naïve Bayes, SVM, Random Forest, logistic and linear regression, neural networks	K-Means, Bisecting K-Means / hierarchical, Gaussian Mixture, DBSCAN, PCA, FP-Growth
Evaluation	Accuracy, precision/recall, F1, AUC, RMSE on a held-out test set	Within-cluster SSE, silhouette score, log-likelihood, visual inspection
Cost of data	Labels are often expensive (human annotation)	Raw data is cheap and plentiful, which suits Big Data
Spark MLlib	`pyspark.ml.classification`, `pyspark.ml.regression`	`pyspark.ml.clustering`, `pyspark.ml.feature.PCA`, `pyspark.ml.fpm`

An analogy: supervised learning is like studying with an answer key. Unsupervised learning is like being handed a pile of documents and asked to sort them into sensible piles without being told what the piles should be. Semi-supervised and reinforcement learning sit between these two.

3.2 Example data

Twenty-five mall customers are described by **annual income (k$)** and **spending score (1–100)**. The data was built with five natural segments of five customers each:

Segment	Customers	Income	Spending
Low income, low spending	1–5	15–23	33–40
Low income, high spending	6–10	16–24	77–85
Mid income, mid spending	11–15	60–65	45–55
High income, high spending	16–20	97–105	85–92
High income, low spending	21–25	96–104	12–22

All three algorithms below run on this data in `code/q3_clustering.py`, with `VectorAssembler` building the feature vector and `ClusteringEvaluator` (silhouette) used for evaluation.

3.3 Algorithm 1 — K-Means (partitional, hard clustering)

Idea. K-Means splits the data into k clusters. It chooses k centroids that minimise the within-cluster sum of squared errors, $WSSSE = \sum_{j=1}^{k} \sum_{x \in C_j} \| x - \mu_j \|^2$, by repeating two steps (Lloyd's algorithm):

1. **Assign**: put each point in the cluster of its nearest centroid (Euclidean distance, from Q1).
2. **Update**: move each centroid to the mean of its assigned points.

These two steps repeat until the assignments stop changing. Spark initialises the centroids with **k-means||**, a parallel version of k-means++, and each iteration is one map (assign) plus one reduce (sum the points per cluster). This makes K-Means a natural fit for MapReduce and Spark.

Hand computation (one iteration, k = 2, four points). The four points are P1(15,39), P2(19,40), P3(60,50) and P4(64,45). The initial centroids are C1 = P1 and C2 = P3.

Point	d(C1)	d(C2)	Assigned
P1 (15,39)	0.00	46.32	C1
P2 (19,40)	√(16+1) = 4.12	√(1681+100) = 42.20	C1
P3 (60,50)	46.32	0.00	C2
P4 (64,45)	√(2401+36) = 49.37	√(16+25) = 6.40	C2

The new centroids are C1 = mean(P1, P2) = **(17.0, 39.5)** and C2 = mean(P3, P4) = **(62.0, 47.5)**. A second iteration would not change any assignment, so the algorithm has converged. The script prints these exact numbers.

Choosing k (elbow method + silhouette), computed with Spark:

k	WSSSE	Silhouette
2	22 543.17	0.6377
3	11 767.33	0.7553

k	WSSSE	Silhouette
4	7 515.21	0.6796
5	**405.20**	**0.9818**
6	361.87	0.8781

WSSSE drops sharply up to k = 5 and then flattens: this is the “elbow”. The silhouette score also peaks at k = 5, so **k = 5**.

Spark result (`KMeans(k=5, seed=1)`): the centroids are (19.0, 37.0), (20.0, 81.0), (62.8, 50.0), (100.6, 88.2) and (100.4, 17.4). The cluster sizes are [5, 5, 5, 5, 5], WSSSE = 405.20 and silhouette = 0.9818. All five segments were recovered perfectly.

Strengths and weaknesses. K-Means is simple, fast ($O(nkt)$) and scales well. However, you must choose k in advance, and it assumes spherical clusters of similar size. It is also sensitive to outliers and to feature scale, and it gives only hard assignments.

3.4 Algorithm 2 — Bisecting K-Means (hierarchical, divisive)

Idea. Bisecting K-Means is a **top-down hierarchical** method:

1. Start with all points in one cluster.
2. Pick a cluster to split (Spark splits divisible clusters level by level, preferring the larger ones).
3. Split it into two with ordinary 2-means.
4. Repeat until there are k leaf clusters or no cluster can be divided.

The result is a binary **tree (dendrogram)** of clusters, so you can cut it at different levels. Each split only runs 2-means on a subset of the data, so it is often faster than K-Means for large k. It is also less dependent on random initialisation, and it tends to produce clusters of more even size.

How the hierarchy unfolds on the example data. The script prints the Spark result for k = 2, 3 and 4. Each split is an ordinary 2-means, like the hand computation in 3.3:

k	Clusters (centre, size)
2	(19.5, 59.0) n=10 · (87.9, 51.9) n=15
3	(19.5, 59.0) n=10 · (81.6, 33.7) n=10 · (100.6, 88.2) n=5
4	(19.0, 37.0) n=5 · (20.0, 81.0) n=5 · (62.8, 50.0) n=5 · (100.5, 52.8) n=10

The first cut follows the direction of greatest spread, income: the 10 low-income customers versus the 15 mid- and high-income customers. The splits that follow separate the remaining segments. Spark divides clusters level by level, so a model with a smaller k is not always an exact prefix of the tree for a larger k. For example, the k = 3 and k = 4 models group the 15 richer customers differently.

Spark result (`BisectingKMeans(k=5, seed=1)`): the centroids are (19.0, 37.0), (20.0, 81.0), (62.8, 50.0), (100.4, 17.4) and (100.6, 88.2). The cluster sizes are [5, 5, 5, 5, 5], SSE = 405.20 and silhouette = 0.9818. On well-separated data it finds the same partition as K-Means, but it does so through a hierarchy.

Strengths and weaknesses. Bisecting K-Means gives a hierarchy and is deterministic given the splits. Bad early splits are never undone, because the splitting is greedy. Classic *agglomerative* (bottom-up) hierarchical clustering needs $O(n^2)$ memory and does not scale, which is why Spark offers the divisive version.

3.5 Algorithm 3 — Gaussian Mixture Model (probabilistic, soft clustering)

Idea. A GMM assumes the data was generated by k Gaussian distributions with mixing weights π_j, means μ_j and covariance matrices Σ_j:

$$p(x) = \sum_{j=1}^{k} \pi_j \, \mathcal{N}(x \mid \mu_j, \Sigma_j)$$

The parameters are estimated by **Expectation-Maximisation (EM)**:

- **E-step**: compute each point's *responsibility*, the probability that it belongs to each component: $\gamma_{ij} = \frac{\pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}{\sum_l \pi_l \mathcal{N}(x_i \mid \mu_l, \Sigma_l)}$.
- **M-step**: re-estimate π_j, μ_j and Σ_j as responsibility-weighted averages.
- Repeat until the log-likelihood stops improving.

Unlike K-Means, every point gets a **probability of membership** in every cluster (soft clustering). The full covariance matrices also allow **elliptical** clusters of different sizes and orientations. K-Means is the special case of a GMM with equal spherical covariances and hard assignments.

Spark result on the customer data (`GaussianMixture(k=5, seed=2)`): the weights are [0.2, 0.2, 0.2, 0.2, 0.2], and the means are (62.8, 50.0), (100.6, 88.2), (19.0, 37.0), (100.4, 17.4) and (20.0, 81.0). The log-likelihood is −160.56 and the silhouette is 0.9818. Because these segments are far apart, every customer's membership probability is about 1.0.

Where the soft assignment matters: overlapping clusters. The script also fits a 2-component GMM to 22 overlapping customer ages (22–50):

	Mean	Variance	Weight
Component 0	27.62	11.85	0.452
Component 1	42.17	22.01	0.548

Age	P(component 0)	P(component 1)
25	0.999	0.001
33	**0.691**	**0.309**

Age	P(component 0)	P(component 1)
35	**0.266**	**0.734**
37	0.048	0.952
47	0.000	1.000

Ages 33 and 35 sit in the overlap. A GMM reports this uncertainty, for example 69% versus 31%, while K-Means would force a single hard label.

Hand check of one E-step value (age 33):

- $\pi_0 \mathcal{N}(33 \mid 27.62, 11.85) = 0.452 \times \frac{1}{\sqrt{2\pi \cdot 11.85}} e^{-(5.38)^2/(2 \cdot 11.85)} = 0.452 \times 0.1159 \times 0.2950 = 0.01545$
- $\pi_1 \mathcal{N}(33 \mid 42.17, 22.01) = 0.548 \times 0.0850 \times e^{-(9.17)^2/44.02} = 0.548 \times 0.0850 \times 0.1480 = 0.00690$
- γ_0 = 0.01545/(0.01545 + 0.00690) = 0.691, which matches Spark.

A practical observation. EM only finds a *local* optimum, so the result depends on the random initialisation. Trying seeds 1–20 on the customer data recovered the five segments perfectly for only seeds 2, 14 and 15. Several other seeds gave a *higher* log-likelihood but worse clusters, because a component collapsed onto very few points. In practice, you run several initialisations, or initialise from a K-Means solution, and compare them with BIC or AIC.

3.6 Comparison of the three algorithms

	K-Means	Bisecting K-Means	Gaussian Mixture
Type	Partitional, centroid-based	Hierarchical (divisive)	Model-based, probabilistic
Assignment	Hard	Hard	**Soft** (probabilities)
Cluster shape	Spherical, similar size	Spherical	Elliptical, any size and orientation
Objective	Minimise WSSSE	Repeated 2-means splits	Maximise likelihood (EM)
Output	k centroids	Tree of clusters + centroids	Weights, means, covariances
Cost	Low	Low to medium	Higher (covariance per component)
Sensitivity	Initialisation, outliers, scale	Early greedy splits	Initialisation, can collapse to a point
Result on example (k=5)	Silhouette 0.9818	Silhouette 0.9818	Silhouette 0.9818 (seed 2)

The script ends with a table of each customer’s assigned cluster under each algorithm. Apart from the arbitrary cluster numbering, all three algorithms produce the same five segments.

Question 4 — Classification

4.1 What is classification?

Classification is a *supervised* learning task. It learns, from labelled training examples (x_i, y_i), a function that assigns a new observation x to one of a fixed set of **discrete classes** $y \in \{c_1, \ldots, c_m\}$. Examples:

- Is an email spam or ham?
- Is a transaction fraudulent?
- Will it rain tomorrow?
- Which digit (0–9) is in this image?

The workflow is:

1. Collect labelled data.
2. Engineer the features (`VectorAssembler`, `StringIndexer`, `OneHotEncoder`, scaling).
3. Split the data into training and test sets.
4. **Train** the model.
5. **Predict** on the test set.
6. **Evaluate** with a confusion matrix, accuracy, precision, recall, F1 and ROC-AUC.

A *binary* problem has two classes and a *multiclass* problem has more. Regression is different: it predicts a *continuous* value.

4.2 The three algorithms in words

Naïve Bayes is a **probabilistic, generative** classifier based on Bayes' theorem:

$$P(c \mid x) = \frac{P(c)\,P(x \mid c)}{P(x)} \;\propto\; P(c)\prod_j P(x_j \mid c)$$

It is "naïve" because it assumes the features are conditionally independent given the class, so the joint likelihood becomes a product of one-feature terms. Training is just counting (for categorical or multinomial data) or estimating a mean and variance per class and feature (Gaussian NB). This takes a single pass over the data, which is extremely fast and scalable. **Laplace smoothing** adds 1 to each count so that an unseen value does not make the whole product zero.

Support Vector Machine (SVM) is a **geometric, discriminative** classifier. It finds the **hyperplane** $w \cdot x + b = 0$ that separates the classes with the **largest margin** $2/\|w\|$. Only the points that lie on the margin, the **support vectors**, determine the boundary. The **soft-margin** version allows some violations through the hinge loss $\max(0, 1 - y(w \cdot x + b))$ plus a regularisation term $\lambda \|w\|^2$. The **kernel trick** (polynomial, RBF) handles non-linear boundaries. Spark MLlib provides a distributed **linear** SVM (`LinearSVC`) trained with OWL-QN.

Random Forest is an **ensemble of decision trees**, an example of bagging:

- Each tree is trained on a **bootstrap sample** (rows drawn with replacement).
- At each split, a tree considers only a **random subset of features**.
- Each split is chosen to reduce impurity the most, measured by Gini $= 1 - \sum_c p_c^2$ or by entropy.
- The forest predicts by **majority vote** of the trees, or by averaging their class probabilities.

The two kinds of randomness make the trees different from one another, so their errors partly cancel out. This reduces variance and overfitting compared with a single tree, and the forest also yields **feature importances**. Spark trains all the trees in parallel.

4.3 Worked example A — "Play Tennis" (hand computation + Spark)

The training data has 14 days with outlook, temperature and humidity, and the label *play* (9 Yes, 5 No). The task is to classify a new day: **Outlook = Sunny, Temp = Cool, Humidity = High**.

Naïve Bayes by hand

	Yes	**No**
Prior P(c)	9/14	5/14
P(Sunny | c)	2/9	3/5
P(Cool | c)	3/9	1/5
P(High | c)	3/9	4/5
Product	9/14 · 2/9 · 3/9 · 3/9 = **0.01587**	5/14 · 3/5 · 1/5 · 4/5 = **0.03429**
Posterior (normalised)	0.3165	**0.6835**

The prediction is **No (don't play)**. Spark computes every conditional count with DataFrame filters and gets exactly these numbers. Spark's `NaiveBayes` (multinomial, one-hot features, Laplace smoothing = 1) gives P(No) = 0.6379, which is also **No**. The probability is slightly different because smoothing pulls the estimates towards uniform.

Random Forest building block: Gini split selection, by hand and on Spark

The root node has 9 Yes and 5 No, so $Gini = 1 - (9/14)^2 - (5/14)^2 = 0.4592$.

Candidate split	**Child nodes (Yes/No)**	**Weighted Gini**	**Gain**
Outlook	Sunny (2/3): 0.48, Overcast (4/0): 0, Rain (3/2): 0.48	**0.3429**	**0.1163**
Humidity	High (3/4): 0.490, Normal (6/1): 0.245	0.3673	0.0918

Candidate split	Child nodes (Yes/No)	Weighted Gini	Gain
Temp	Hot (2/2): 0.5, Mild (4/2): 0.444, Cool (3/1): 0.375	0.4405	0.0187

A single tree would split on **Outlook** first. In a Random Forest, each tree only sees a bootstrap sample and a random subset of features, so different trees split differently. A 7-tree `RandomForestClassifier` on Spark voted **[No, No, Yes, Yes, No, No, No]**: 5 votes against 2 for **No**. The forest's probability is P(No) = 0.701, which agrees with Naïve Bayes.

SVM by hand on a tiny 2-D set

The class −1 points are (1,1), (2,1) and (1,2). The class +1 points are (3,3), (4,3) and (3,4). By symmetry $w = (a, a)$. The closest points are (2,1) and (1,2) on the negative side ($x_1 + x_2 = 3$) and (3,3) on the positive side ($x_1 + x_2 = 6$), so they are the support vectors. Requiring $w \cdot x + b = -1$ at the negative support vectors and +1 at the positive one gives:

- $3a + b = -1$ and $6a + b = +1$
- so $a = 2/3$ and $b = -3$
- The boundary is $\frac{2}{3}x_1 + \frac{2}{3}x_2 - 3 = 0$, that is $x_1 + x_2 = 4.5$.
- The margin is $2/\|w\| = 2/0.9428 = 2.1213$, which equals the distance between the lines $x_1 + x_2 = 3$ and $x_1 + x_2 = 6$, $3/\sqrt{2}$.

Spark's `LinearSVC` gives $w = (0.7958, 0.7958)$ and $b = -3.7749$:

- The direction of w is the same as the hand solution, and every point is classified correctly.
- The support vector (3,3) lies exactly on the margin, with $w \cdot x + b = +1.0000$.
- The boundary sits slightly off centre ($x_1 + x_2 \approx 4.74$).

The offset arises because `LinearSVC` minimises a regularised hinge loss with a quasi-Newton optimiser (OWL-QN). That only approximates the exact quadratic program that a textbook SVM solves.

4.4 Worked example B — spam detection with all three classifiers on Spark

Data. 400 synthetic emails, 200 spam and 200 ham, generated with a fixed random seed. Each email has three features:

- the number of links
- the count of "spammy" words (free, win, …)
- the fraction of the text in CAPITALS

class	mean links	mean spam words	mean caps ratio
ham (0)	1.90	1.08	0.104

class	mean links	mean spam words	mean caps ratio
spam (1)	6.01	3.98	0.358

The data is split 70/30 (`randomSplit(seed=42)`) into 274 training rows and 126 test rows. All three models are Spark ML `Pipelines`:

```
        NaiveBayes(modelType="gaussian")
        Pipeline([StandardScaler(withMean=True, withStd=True),
LinearSVC(regParam=0.01)])
        RandomForestClassifier(numTrees=100, maxDepth=5, seed=42)
```

Test-set results (from `outputs/q4_classification.txt`):

Model	Accuracy	F1	Precision (spam)	Recall (spam)	ROC-AUC
Naïve Bayes (Gaussian)	**0.9762**	**0.9762**	0.9683	**0.9839**	0.9798
Linear SVM	0.9683	0.9683	0.9677	0.9677	**0.9972**
Random Forest (100 trees)	0.9444	0.9444	0.9104	**0.9839**	0.9961

Confusion matrices (rows = actual, columns = predicted):

	NB: ham	NB: spam	SVM: ham	SVM: spam	RF: ham	RF: spam
actual ham (64)	62	2	62	2	58	6
actual spam (62)	1	61	2	60	1	61

What each fitted model learned:

- **Naïve Bayes** stores, per class, a log-prior (−0.7005 for ham and −0.6859 for spam) and a mean and variance for each feature. For example, the mean number of links is 2.11 for ham and 6.10 for spam. A new email is scored by multiplying the three Gaussian likelihoods with the prior.
- **SVM** learned one hyperplane on the standardised features: w = (0.886, 1.178, 1.118) and b = 0.308. The sign of $w \cdot x + b$ gives the class, and its magnitude is a confidence. That is why the SVM has the best AUC.
- **Random Forest** reports feature importances: caps ratio 0.563, spam words 0.259, links 0.178. This tells us *which* signals matter most, which the other two models do not show as directly.

Predictions for three new emails:

links	spam words	caps	NB	SVM	RF
1	0	0.05	ham	ham	ham

links	spam words	caps	NB	SVM	RF
4	2.5	0.22	spam	spam	**ham**
8	6	0.50	spam	spam	spam

All three models agree on the clear cases. The borderline email splits them: NB and SVM use smooth boundaries, while the forest uses axis-aligned thresholds. With `maxDepth=5`, the forest's thresholds put 0.22 caps and 4 links on the ham side.

Interpretation. On this nearly Gaussian, nearly linearly separable data, the simple models (NB and the linear SVM) are as good as or better than the forest. Their assumptions match how the data was generated. Random Forest becomes the stronger choice when boundaries are non-linear, when features interact, or when features are mixed categorical and numeric.

4.5 Distinguishing Naïve Bayes, SVM and Random Forest

Aspect	Naïve Bayes	SVM	Random Forest
Family	Probabilistic, **generative** (models $P(x \mid c)$)	Geometric, **discriminative** (models the boundary)	**Ensemble** of decision trees (bagging)
Core principle	Bayes' theorem + conditional independence	Maximum-margin separating hyperplane	Majority vote of de-correlated trees
Decision boundary	Linear (multinomial) or quadratic (Gaussian)	Linear; non-linear with kernels	Piecewise axis-aligned, highly non-linear
Training	One pass of counting (very fast)	Convex optimisation (iterative)	Grow many trees in parallel (heavier)
Key hyperparameters	Smoothing, model type	C / regParam, kernel	numTrees, maxDepth, featureSubsetStrategy
Output	Calibrated-ish class probabilities	Signed distance / score (no native probability)	Vote fractions as probabilities + feature importance
Feature scaling needed?	No	**Yes** (distance-based)	No
Handles feature interactions	No (independence assumption)	Only via kernels	**Yes**, naturally
Robustness	Works with little data; hurt by correlated features	Robust in high dimensions; sensitive to C and scaling	Robust to outliers and noise; resists overfitting
Interpretability	High (inspect probabilities)	Medium (weights for linear)	Medium (feature importances, but many trees)
Best for	Text and spam, real-time, very high-dimensional sparse data	High-dimensional data with a clear margin (text, bioinformatics)	Tabular data with mixed types and non-linear effects
Spark MLlib class	`NaiveBayes` (multinomial, Bernoulli,	`LinearSVC` (binary, linear)	`RandomForestClassifier`

Aspect	Naïve Bayes	SVM	Random Forest
	Gaussian, complement)		
Result on Play Tennis query	No (0.684)	—	No (5 of 7 trees)
Spam test accuracy	0.976	0.968	0.944

References

1. Apache Spark. *MLlib: Main Guide — Basic Statistics (Correlation, Hypothesis testing, ChiSquareTest), Clustering, Classification and Regression, Feature Extraction (MinHash LSH)*. Spark 4.0 documentation. https://spark.apache.org/docs/latest/ml-guide.html
2. Apache Spark. *PySpark API Reference — pyspark.ml*. https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html
3. Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of Massive Datasets* (3rd ed.), Ch. 3 (Finding Similar Items, Jaccard, MinHash, LSH) and Ch. 7 (Clustering). Cambridge University Press. http://www.mmds.org
4. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.), Ch. 2 (Measuring Data Similarity and Dissimilarity), Ch. 8–9 (Classification), Ch. 10 (Cluster Analysis). Morgan Kaufmann.
5. Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2018). *Introduction to Data Mining* (2nd ed.). Pearson. Bisecting K-Means: §7.2.3.
6. Montgomery, D. C., & Runger, G. C. (2014). *Applied Statistics and Probability for Engineers* (6th ed.), Ch. 9–10 (Tests of Hypotheses, χ^2, t, F) and Ch. 13 (ANOVA). Wiley.
7. Mitchell, T. M. (1997). *Machine Learning*, Ch. 6 (Bayesian Learning; the “Play Tennis” example). McGraw-Hill.
8. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
9. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
10. Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society B*, 39(1), 1–38.
11. Virtanen, P. et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. (Used to cross-check p-values.)

All example datasets in this report are small illustrative datasets constructed for this assignment. The “Play Tennis” layout follows Mitchell (1997), with the Wind attribute omitted.